

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.tsa.holtwinters import ExponentialSmoothing

from sklearn.metrics import mean_squared_error
```

1. Load & Preprocess

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv('/content/Electric_Production.csv')
# Correct column names
df.columns = ['DATE', 'Production']
# Convert date
df['DATE'] = pd.to_datetime(df['DATE'])
df.set_index('DATE', inplace=True)
# SET FREQUENCY (VERY IMPORTANT)
df = df.asfreq('MS')

print(df.head())
```

DATE	Production
1985-01-01	72.5052
1985-02-01	70.6720
1985-03-01	62.4502
1985-04-01	57.4714
1985-05-01	55.3151

```
plt.figure(figsize=(12,5))
plt.plot(df['Production'])
plt.title('Electric Production Over Time')
plt.show()
```

[Show hidden output](#)

2. Seasonality Proof

Decomposition

```
from statsmodels.tsa.seasonal import seasonal_decompose
decomp = seasonal_decompose(df['Production'], model='multiplicative')
decomp.plot()
plt.show()
```

[Show hidden output](#)

Monthly Pattern

```
df['month'] = df.index.month
monthly_avg = df.groupby('month')['Production'].mean()
monthly_avg.plot(kind='bar', figsize=(8,5))
plt.title('Average Production per Month')
plt.show()
```

[Show hidden output](#)

4. Stationarity Check

```
from statsmodels.tsa.stattools import adfuller
#p > 0.05 → Not stationary
result = adfuller(df['Production'])
print("p-value:", result[1])
```

p-value: 0.1862146911658712

Differencing

```
df_diff = df['Production'].diff().dropna()
plt.plot(df_diff)
plt.title('Differenced Data')
plt.show()
```

[Show hidden output](#)

```
train = df.iloc[:-12]
test = df.iloc[-12:]
```

6. APPLY MODELS

AR

```
import pandas as pd
from statsmodels.tsa.ar_model import AutoReg

model_ar = AutoReg(train['Production'], lags=12).fit()

forecast_ar = model_ar.predict(start=len(train), end=len(train)+len(test)-1)
```

Moving Average

```
ma_val = train['Production'].rolling(12).mean().iloc[-1]
forecast_ma = [ma_val] * len(test)
```

ARIMA

```
from statsmodels.tsa.arima.model import ARIMA

model_arima = ARIMA(train['Production'], order=(2,1,2)).fit()
forecast_arima = model_arima.forecast(len(test))
```

SARIMA

```
from statsmodels.tsa.statespace.sarimax import SARIMAX

model_sarima = SARIMAX(train['Production'], order=(1,1,1), seasonal_order=(1,1,1,12)).fit()
forecast_sarima = model_sarima.forecast(len(test))
```

Holt-Winters

```
from statsmodels.tsa.holtwinters import ExponentialSmoothing

model_hw = ExponentialSmoothing(train['Production'], seasonal='multiplicative', seasonal_periods=12).fit()

forecast_hw = model_hw.forecast(len(test))
```

7. AIC & BIC Comparison

```
print("AR AIC:", model_ar.aic)
print("ARIMA AIC:", model_arima.aic)
```

```

print("SARIMA AIC:", model_sarima.aic)
print("Holt-Winters AIC:", model_hw.aic)

print('-----')

print("AR BIC:", model_ar.bic)
print("ARIMA BIC:", model_arima.bic)
print("SARIMA BIC:", model_sarima.bic)
print("Holt-Winters BIC:", model_hw.bic)

```

```

AR AIC: 1816.8259085613295
ARIMA AIC: 2011.4541640073824
SARIMA AIC: 1702.0190819498216
Holt-Winters AIC: 690.7699715592963
-----
AR BIC: 1871.7280064363429
ARIMA BIC: 2031.207376770321
SARIMA BIC: 1721.6135512211874
Holt-Winters BIC: 746.1153782393253

```

8. Evaluation :RMSE & MAE

```

from sklearn.metrics import mean_squared_error
rmse_ma = np.sqrt(mean_squared_error(test['Production'], forecast_ma))
rmse_ar = np.sqrt(mean_squared_error(test['Production'], forecast_ar))
rmse_arima = np.sqrt(mean_squared_error(test['Production'], forecast_arima))
rmse_sarima = np.sqrt(mean_squared_error(test['Production'], forecast_sarima))
rmse_hw = np.sqrt(mean_squared_error(test['Production'], forecast_hw))

print("AR RMSE:", rmse_ar)
print("Moving Avg:", rmse_ma)
print("ARIMA:", rmse_arima)
print("SARIMA:", rmse_sarima)
print("Holt-Winters:", rmse_hw)
print("Moving Avg:", rmse_ma)

```

```

AR RMSE: 5.4524412197164605
Moving Avg: 10.982412637817415
ARIMA: 6.556317211801126
SARIMA: 4.6161851878145175
Holt-Winters: 4.969390680878857
Moving Avg: 10.982412637817415

```

```

plt.figure(figsize=(12,6))

plt.plot(train['Production'], label='Train')
plt.plot(test['Production'], label='Actual')

plt.plot(test.index, forecast_arima, label='ARIMA')
plt.plot(test.index, forecast_sarima, label='SARIMA')
plt.plot(test.index, forecast_hw, label='Holt-Winters')
plt.plot(test.index, forecast_ma, label='Moving Avg')

plt.legend()
plt.title('Model Comparison - Electric Production')
plt.show()

```

[Show hidden output](#)

9. Best Model Selection

```

results = {
    "AR": rmse_ar,
    "Moving Avg": rmse_ma,
    "ARIMA": rmse_arima,
    "SARIMA": rmse_sarima,
    "Holt-Winters": rmse_hw,
}

best_model = min(results, key=results.get)
print("Best Model:", best_model)

```

Best Model: SARIMA